



Cofinanciado por
la Unión Europea



GOBIERNO
DE ESPAÑA

MINISTERIO
DE INDUSTRIA
Y TURISMO



Escuela de
organización
Industrial



Fondos Europeos



Chaticon

Curso de Programación Python para IA

Índice

1.	Introducción.....	3
2.	Descripción del proyecto	3
3.	Tecnologías utilizadas	3
4.	Estructura del código	4
4.1	Archivo main.py	4
4.2	Archivo emojis.py.....	5
4.3	Archivo tiempo.py	6
4.4	Archivo traductor.py.....	6
4.5	Archivo index.js	7
4.6	Archivo variables.js	8
4.7	Archivo styles.css	8
4.8	Archivo emojis.csv.....	8
4.9	Archivo index.html	8
5.	Funcionamiento del sistema.....	9
6.	Capturas de pantalla	10
7.	Consideraciones adicionales.....	12
8.	Bibliografía	12
9.	Conclusión	14

1. Introducción

Este proyecto tiene como objetivo desarrollar una aplicación que genere iconos a partir del texto que el usuario introduzca por el prompt en tiempo real. Además, el usuario será capaz de consultar el tiempo de la ciudad que desee o traducir los mensajes antes escrito por el prompt que se muestren en pantalla.

2. Descripción del proyecto

El proyecto se compone de una aplicación web que mediante un csv sugiere emojis basados en palabras clave proporcionadas por el usuario. Por otra parte, la web interactúa con la API de OpenWeather para obtener el tiempo de la ciudad que el usuario escribe en un input, y por último, se utilizará el modelo TFMarianMTModel para hacer traducciones de los textos introducidos por el usuario de español a inglés y viceversa.

3. Tecnologías utilizadas

- **Python y Flask:** para el backend y el servidor web.
- **JavaScript, HTML y CSS:** para la interfaz de usuario y la interacción con el backend.
- **TFMarianMTModel:** permite realizar traducciones automáticas.
- **MarianTokenizer:** prepara y tokeniza el texto de entrada para que sea procesado por TFMarianMTModel.
- **Matplotlib:** para crear gráficos y visualizaciones a partir de datos.
- **BytesIO:** para manejar datos en memoria.
- **MultinomialNB:** para clasificar textos y sugerir emojis en función de las palabras clave.
- **TfidfVectorizer:** para el aprendizaje automático de modelos en vectores.
- **Pandas:** para cargar, manipular y analizar datos guardados en csv.
- **SnowballStemmer:** para preprocesar el texto eliminando sufijos.

- **Flask:** facilita la creación de rutas para manejar solicitudes HTTP.
- **OpenAI:** para integrar modelos de lenguaje avanzado, generación de texto, o comprensión de texto natural dentro de la aplicación.

4. Estructura del código

4.1 Archivo main.py

- **Descripción:** Este archivo implementa un servidor utilizando Flask para manejar diferentes tipos de solicitudes, incluyendo traducción de texto a emojis, obtención de información meteorológica, traducción de textos entre español e inglés, y generación de respuestas mediante un modelo de lenguaje de OpenAI. Utiliza bibliotecas como Pandas para el procesamiento de datos y CORS para permitir acceso desde cualquier origen.
- **Funcionamiento:**
 - Configuración del Servidor y CORS: Se inicializa una aplicación Flask con soporte para CORS para permitir solicitudes de distintos dominios.
 - Rutas Definidas:
 - /emojis: Traduce texto a emojis y devuelve la respuesta en JSON.
 - /tiempo: Obtiene datos meteorológicos de una ciudad específica y devuelve estadísticas como temperatura y humedad.
 - /tiempo/grafica: Genera y devuelve un gráfico basado en datos meteorológicos de una ciudad.
 - /traductor: Traduce texto entre español e inglés basado en un parámetro de idioma.

- /chat: Interactúa con un modelo de OpenAI para generar respuestas de chat. Procesa los títulos de las noticias, limpia los textos eliminando *stopwords* y caracteres no deseados.
- Manejo de Solicitudes: Cada ruta procesa los parámetros recibidos, llama a funciones específicas para realizar acciones (como traducción o generación de gráficos), y devuelve la respuesta correspondiente en formato JSON o como archivo.
- Ejecución del Servidor: El servidor se ejecuta en el puerto 5000, permitiendo que las aplicaciones cliente se conecten y realicen operaciones definidas.

4.2 Archivo emojis.py

- **Descripción:** implementa un modelo de procesamiento de lenguaje natural para traducir texto a emojis utilizando técnicas de aprendizaje automático. Emplea bibliotecas como Pandas para el manejo de datos, Scikit-learn para el modelado y NLTK para la preprocesamiento de texto.
- **Funcionamiento:**
 - La función preprocesar_texto limpia el texto eliminando caracteres especiales, convierte el texto a minúsculas, tokeniza las palabras, elimina las palabras vacías (stopwords) y aplica la lematización con un stemmer.
 - La función entrenar_modelo carga datos de un archivo CSV, elimina filas con emojis duplicados, y preprocesa las palabras clave.
 - La función traducir_a_emojis toma un texto de entrada y lo preprocesa.
 - Se carga y entrena el modelo modelo_emojis utilizando un archivo CSV de datos de entrenamiento localizado en csv/emojis.csv.

4.3 Archivo tiempo.py

- **Descripción:** contiene funciones que interactúan con la API de OpenWeather para obtener datos meteorológicos de una ciudad específica y generar gráficos de temperatura usando matplotlib.
- **Funcionamiento:**
 - La función `obtener_datos_tiempo(ciudad)` realiza una solicitud HTTP a la API de OpenWeather para obtener el pronóstico del tiempo para una ciudad específica.
 - La función `generar_grafico_tiempo(datos, ciudad)` toma los datos del pronóstico y extrae las fechas y las temperaturas correspondientes.

4.4 Archivo traductor.py

- **Descripción:** define un sistema de traducción automatizada entre inglés y español utilizando modelos preentrenados de la biblioteca Transformers de Hugging Face. Se configuran dos modelos, uno para traducir del español al inglés y otro para traducir del inglés al español.
- **Funcionamiento:**
 - `traducir_a_ingles(texto)`: Toma un texto en español como entrada, lo tokeniza utilizando `tokenizer_es_en`, y lo traduce al inglés usando `model_es_en`. La traducción se decodifica y se devuelve como una cadena de texto en inglés.
 - `traducir_a_espanol(texto)`: Similar a la función anterior, pero para traducir un texto de inglés a español. Utiliza `tokenizer_en_es` para tokenizar el texto en inglés y `model_en_es` para generar la traducción en español.

4.5 Archivo index.js

- **Descripción:** gestiona la interfaz de usuario para traducir texto a emojis, mostrar mensajes, obtener datos meteorológicos, y traducir textos entre inglés y español.
- **Funcionamiento:**
 - emojisData(): Configura un manejador de eventos para el campo de texto que traduce el contenido a emojis después de un breve retraso.
 - traducirEmojis(texto): Envía el texto al servidor para obtener una lista de emojis correspondientes y actualiza la interfaz con estos emojis. Si el campo de texto está vacío, oculta la sección de emojis.
 - addMessage(): Configura el botón de envío para agregar mensajes a un área de mensajes. Los mensajes pueden ser traducidos entre inglés y español al hacer clic en los botones correspondientes.
 - obtenerTiempo(ciudad): Obtiene datos meteorológicos actuales y los muestra en la página.
 - obtenerEstadisticas(ciudad): Solicita estadísticas meteorológicas y las muestra en la página.
 - obtenerGrafica(ciudad): Obtiene y muestra un gráfico del tiempo.
 - tiempoData(): Configura un manejador de eventos para el campo de ciudad que actualiza los datos meteorológicos, las estadísticas y el gráfico en función de la ciudad ingresada.
 - showTiempo(): Configura botones para mostrar y ocultar la información meteorológica en la interfaz.
 - traducirTexto(): Traduce el texto de un mensaje al idioma especificado y actualiza el contenido del mensaje en la interfaz.

4.6 Archivo variables.js

- **Descripción:** define constantes y variables globales utilizadas en el resto del script para interactuar con servicios backend y APIs externas.
- **Funcionamiento:**
 - URL_PYTHON se usa para construir URLs para hacer peticiones al backend Flask, como la traducción de texto y la obtención de datos meteorológicos.
 - API_KEY_OPEN_WEATHER es la clave para autenticar peticiones a OpenWeatherMap.
 - URL_OPEN_WEATHER se usa para solicitar datos meteorológicos actuales para una ciudad específica.
 - timeout se usa para evitar múltiples solicitudes simultáneas mientras el usuario escribe, gestionando el retraso antes de hacer la petición para traducir texto a emojis.

4.7 Archivo styles.css

- **Descripción:** proporciona la apariencia y el diseño de la interfaz de usuario para una aplicación web. Define la presentación visual de los elementos HTML, incluyendo el formato, el color, la disposición y otros aspectos estéticos para lograr una experiencia de usuario atractiva y coherente.

4.8 Archivo emojis.csv

- **Descripción:** contiene un conjunto de datos que relaciona palabras clave con sus correspondientes emojis. Este archivo se utiliza para entrenar un modelo de Machine Learning que puede traducir textos a emojis basándose en palabras clave. Cada fila en el archivo representa una entrada de datos que asocia una palabra clave con uno o más emojis.

4.9 Archivo index.html

- **Descripción:** define la estructura básica de la página web, incluyendo la interfaz de usuario para traducir texto a emojis y gestionar mensajes. Contiene formularios para la entrada de texto, visualización de emojis, y consultas sobre el tiempo, junto con elementos para mostrar traducciones y gráficos de datos meteorológicos.

5. Funcionamiento del sistema

1. Entrada de Texto y Emojis:

- El usuario ingresa texto en un campo de entrada en la interfaz web.
- Mientras el usuario escribe, el sistema envía el texto a un servidor Flask para traducirlo a emojis.
- El servidor responde con los emojis correspondientes, que se muestran en la interfaz.

2. Gestión de Mensajes:

- El usuario puede agregar mensajes a un área específica en la interfaz web.
- Cada mensaje puede ser traducido entre inglés y español al hacer clic en los botones correspondientes.
- El sistema envía el mensaje al servidor para traducirlo y actualiza la interfaz con la traducción.

3. Consulta de Información Meteorológica:

- El usuario ingresa una ciudad en el campo de texto correspondiente.
- Al ingresar o modificar la ciudad, se envía una solicitud al servidor para obtener datos meteorológicos de esa ciudad.
- El servidor responde con datos como temperatura actual, condiciones meteorológicas, y estadísticas relacionadas.

4. Generación y Visualización de Gráficos:

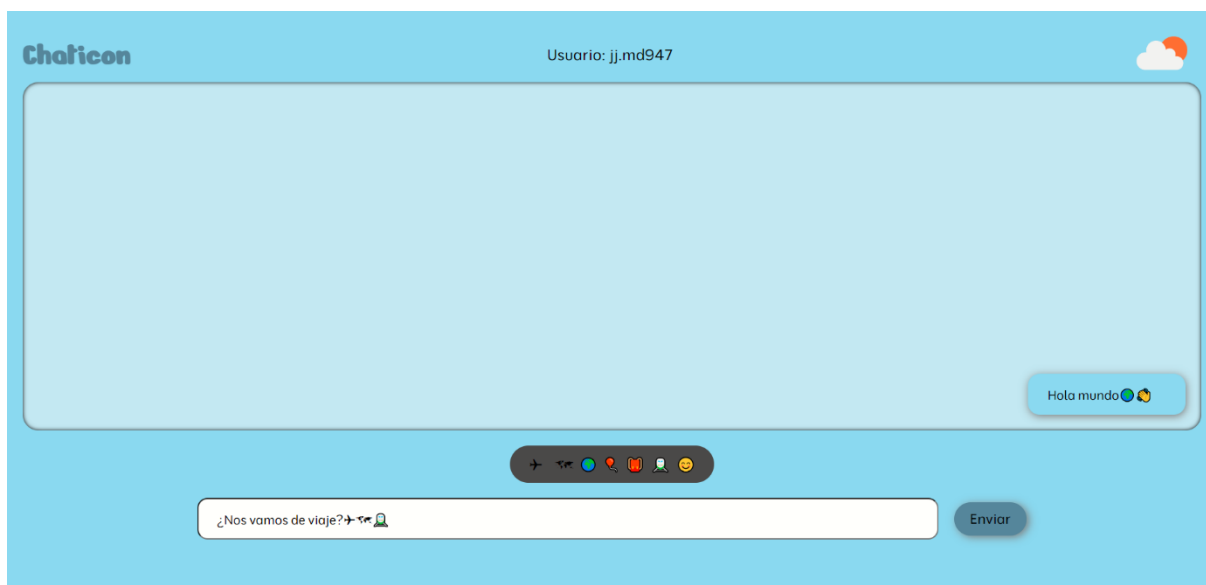
- La información meteorológica obtenida se utiliza para generar gráficos que muestran las variaciones de temperatura y otras estadísticas.
- Los gráficos se crean en el servidor y se envían de vuelta al cliente para ser mostrados en la interfaz web.

5. Configuración y Carga Inicial:

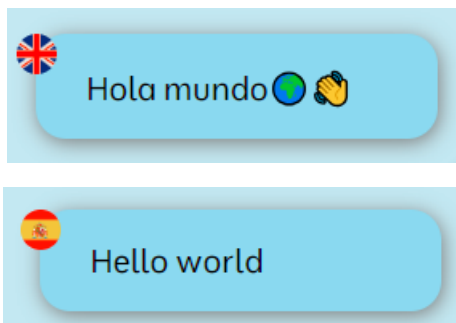
- Al cargar la página, se configuran los eventos y acciones necesarias, como la escucha de entradas de texto y clics en botones.
- Se obtienen datos iniciales de tiempo para la ciudad predeterminada, y se muestran los gráficos y estadísticas correspondientes.

6. Capturas de pantalla

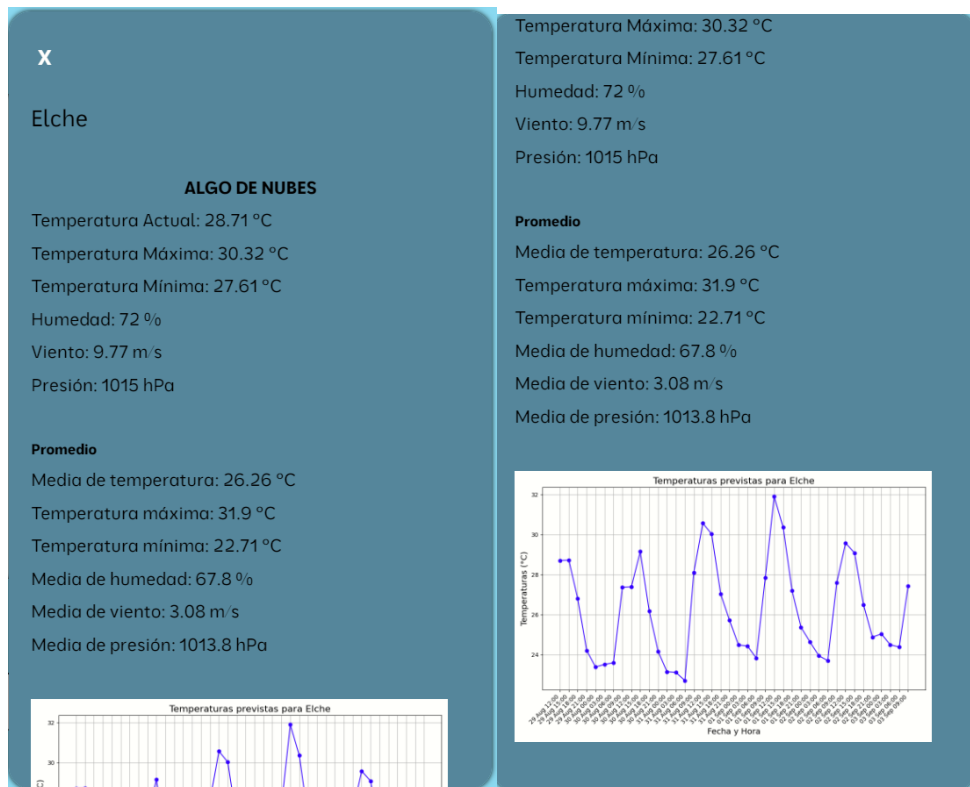
- Interfaz completa



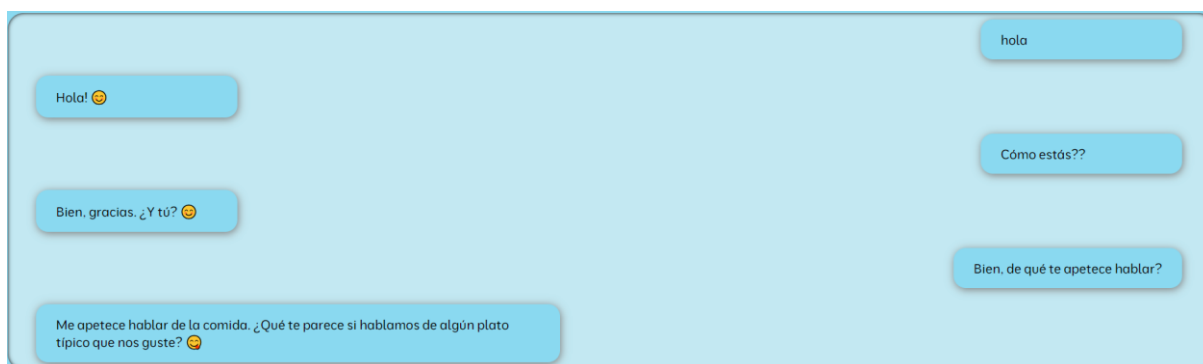
- Botón para traducir los textos de inglés a español y viceversa



- Menú del tiempo



- Chat en tiempo real con IA



7. Consideraciones adicionales

- **Seguridad:** La aplicación utiliza una clave API que debe mantenerse segura y fuera del alcance público.
- **Rendimiento:** Optimiza la generación de gráficos y traducción de texto para reducir tiempos de espera.
- **Experiencia del Usuario:** interfaz fácil de usar e intuitiva.

8. Bibliografía

- Flask: Microframework de Python utilizado para construir aplicaciones web.

<https://flask.palletsprojects.com/en/3.0.x/>

- Transformers (Hugging Face): Biblioteca de Python para modelos de traducción y procesamiento de lenguaje natural.

<https://huggingface.co/docs/transformers/index>

- TensorFlow: Framework de aprendizaje automático utilizado en modelos de traducción automática.

<https://www.tensorflow.org/?hl=es>

- Pandas: Biblioteca de Python para el manejo y análisis de datos en estructuras tabulares.

<https://pandas.pydata.org/>

- Matplotlib: Biblioteca de Python para la creación de gráficos estáticos, animados e interactivos.

<https://matplotlib.org/>

- BytesIO: Módulo de Python utilizado para manejar flujos de datos en memoria como archivos.

<https://docs.python.org/3/library/io.html#io.BytesIO>

- Scikit-learn: Biblioteca de Python para el aprendizaje automático y el procesamiento de datos.

<https://scikit-learn.org/stable/>

- NLTK (Natural Language Toolkit): Librería de Python para el procesamiento de lenguaje natural.

<https://www.nltk.org/>

- OpenWeatherMap API: API para obtener datos meteorológicos en tiempo real.

<https://openweathermap.org/api>

- JavaScript Fetch: Proporciona una forma sencilla de realizar solicitudes HTTP asíncronas en JavaScript.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

- CORS (Cross-Origin Resource Sharing): Mecanismo que permite solicitudes controladas desde un dominio diferente.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

- HTML5 y JavaScript: Tecnologías base para la construcción de interfaces web modernas.

<https://developer.mozilla.org/en-US/docs/Glossary/HTML5>

<https://developer.mozilla.org/en-US/docs/Web/JavaScript>

9. Conclusión

Este proyecto demuestra cómo diversas librerías y tecnologías pueden combinarse para crear herramientas interactivas y valiosas para los usuarios. Utilizando Python y Flask, se ha desarrollado un backend eficiente capaz de manejar solicitudes de los usuarios, interactuar con APIs externas y procesar datos de texto para producir visualizaciones útiles.

Python se destaca como una opción excelente para este proyecto debido a varias razones clave:

- **Integración Sencilla y Uso de Librerías Especializadas:** Python proporciona una amplia gama de librerías, como transformers y nltk, que facilitan el procesamiento del lenguaje natural y la creación de visualizaciones. Estas librerías permiten realizar tareas complejas con menor esfuerzo, respaldadas por una documentación completa y un soporte activo.
- **Claridad y Legibilidad del Código:** La sintaxis de Python es clara y fácil de entender, lo que acelera el desarrollo y reduce la posibilidad de errores. Esta característica es crucial para proyectos que abarcan múltiples etapas, desde la recolección y limpieza de datos hasta la visualización y análisis de resultados.
- **Versatilidad de Flask como Framework Web:** Flask es un microframework de Python que, aunque ligero, resulta muy potente para construir aplicaciones web. Su capacidad para manejar solicitudes HTTP, archivos estáticos y contenido dinámico lo convierte en una opción ideal tanto para prototipos rápidos como para despliegues eficientes.
- **Eficiencia en el Manejo de Datos y Automatización de Procesos:** Python es conocido por su habilidad para gestionar grandes volúmenes de datos y su capacidad para integrarse con APIs y servicios web. En este proyecto, Python se utilizó para automatizar la recopilación, procesamiento y visualización de datos, mejorando así la experiencia del usuario.
- **Comunidad Activa y Amplio Soporte:** La gran y activa comunidad de Python asegura que cualquier problema durante el desarrollo pueda ser resuelto rápidamente, gracias a la abundancia de recursos y foros de soporte disponibles.



Cofinanciado por
la Unión Europea



Fondos Europeos



En resumen, Python no solo ofrece las herramientas necesarias para desarrollar un flujo de trabajo sólido y escalable, sino que también facilita la rápida iteración y mejora del proyecto. Su uso en esta aplicación, que traduce texto a emojis, visualiza el clima y traduce textos, está plenamente justificado por su versatilidad, facilidad de uso y el extenso ecosistema de librerías que soportan cada fase del desarrollo.